ARFAPOS Clone — Functional Spec & Build Plan

Oct 10, 2026 · @Dev

Purpose and scope

This is a build-ready functional spec of ARFAPOS, taken from a read-only review of the live admin panel at bizpos.airoxlab.com on 10 October 2026, so that an equivalent system can be built in-house.

ARFAPOS is a subscription restaurant management platform. The reviewed account runs on its Business plan for a single restaurant with 183 products counted on the dashboard (155 listed in the default menu), 891 customers and roughly 240 orders a day. The admin panel carries 12 module groups and about 80 screens, covering far more than point of sale: full inventory with recipe-level deduction, purchasing and supplier ledgers, double-entry finance with a chart of accounts and balance sheet, payroll with fingerprint attendance, delivery riders, loyalty and vouchers, and an AI layer for stock forecasting and pricing.

Target stack for the clone

Layer	Choice
Backend	Ruby on Rails 8, API-only
Database	PostgreSQL 16
Frontend	Next.js 15 (App Router) + React 19, TypeScript
Realtime	ActionCable for kitchen and order screens
Background jobs	Sidekiq + Redis
Auth	Devise + JWT, role-based scopes per outlet

What this spec covers and what it does not

It documents behaviour: screens, fields, workflows, rules and reports. Features are not protected, so a system that does the same jobs is fair to build. It does not copy ARFAPOS source code, templates, stylesheets, wording or branding, and the clone should carry its own name, visual design and copy.

No data was created, edited or deleted during the review. Screens that could only be confirmed by writing data are listed as open questions at the end.

System map

The admin panel is organised into 12 collapsible module groups plus four standalone screens. Every route below was read from the live sidebar, and each is a screen the clone needs an equivalent of.

Group	Screens
Standalone	Dashboard · Orders · Expenses · Tables
Products	Menus · Catalog · Deals · Promotions · Quick Add Menu
Inventory	Stock in Hand · Stock Actions · Demand · Items · Recipes · Order Inventory · Units · Inventory Settings
Inventory Reports	Daily Stock Sheet · Stock Ledger · Consumption · Waste & Adjustments · Actual vs Theoretical · Valuation & COGS · AI Stock Forecast · Deduction Health
Purchasing	Purchase Orders · Purchase Returns · Purchase Price History
Suppliers	Suppliers · Supplier Ledger · Supplier Insights · Supplier Payments
Customers	Customers · Customer Ledger · Customer Insights · Customer Payments · Customer Account · AR Aging
Staff	Cashiers · User Sessions · Order Takers · Admin Staff
Finance	Dashboard · Payment Accounts · Combined Ledger · Journal Voucher · Chart of Accounts · Fixed Assets · Liabilities · Equity · Balance Sheet · Trial Balance
Payroll	Overview · Employees · Advances · Run Payroll · Ledger · Fingerprint · Reports · Settings

Group	Screens
Reports	Business Overview · Grand Report · Sales · Sales Calendar · Financial · Products & Deals · Category Report · Order Takers · Cashiers · Cashier Till · Customers · Customer Ledger · Operations · Cancelled & Modified Orders · Delivery Riders · Vouchers · Loyalty Points · Regular Discounts · AI Pricing
Riders	Manage Riders · Rider Vault · Rider Reports · Rider App
Settings	Store · Payment Methods · Plan · WhatsApp · Website Config · Notifications · Loyalty Points · Regular Discounts · Vouchers · Reviews · Audit Logs

Three things stand out for the build. Inventory is recipe-driven, so selling a product deducts raw ingredients and the system then reports theoretical against actual stock. Finance is real double-entry, not a sales summary, with a chart of accounts, journal vouchers and a balance sheet. Separate client apps exist beyond the admin: a cashier POS, an order-taker screen, a rider app and a customer-facing website driven by Website Config.

Roles and permissions

Permissions are granted per user, not per role: 58 individual permissions across 12 modules, with role presets as a shortcut. This is the single most detailed part of the system and the part most worth copying carefully.

Staff populations are separate record types, each with its own screen and its own login:

Population	Screen	Notes
Admin Staff	Staff → Admin Staff	Back-office users of this admin panel
Cashiers	Staff → Cashiers	Roles: Cashier, Supervisor, Kitchen Display. Carry sessions and tills
Order Takers	Staff → Order Takers	Floor staff who punch walk-in orders; tracked for daily performance

Population	Screen	Notes
Riders	Riders → Manage Riders	Permanent riders, plus throwaway temporary riders with a lifetime
Employees	Payroll → Employees	HR and salary records, linked to fingerprint attendance

How the permission editor works. Opening a cashier's access shows a full-screen editor: a module list down the left with a granted-of-total badge per module, the permissions for the selected module on the right, and three auto-select presets at the top (Cashier, Supervisor, Manager) plus Select all. Each permission carries a machine code, a plain-English description and a surface badge:

- **Software** — takes effect on the desktop POS only
- **App** — takes effect on the tablet app only
- **Both** — one shared setting; it cannot be granted on one surface and withheld on the other

Some permissions are marked RESTRICTION, meaning ticking them adds a step rather than opening a door. Changes take effect at the user's next sign-in, not immediately.

The permission set

Module	Permissions
Dashboard (6)	DASHBOARD · VIEW_DAILY_SALES · VIEW_SALES_ANALYTICS · OFFLINE_ORDERS · PRINTERS · SETTINGS
Take Orders (3)	SALES_WALKIN · SALES_TAKEAWAY · SALES_DELIVERY
Orders (8)	ORDERS · WEB_ORDERS · COMPLETE_ORDER · CANCEL_ORDER · REOPEN_ORDER · PLACE_AND_COMPLETE_ORDER · COMPLIMENTARY_ORDER
Editing a Reopened Order (2)	DELETE_ORDER_ITEM · MODIFY_REOPEN_DECREASE_QTY
Payment (4)	PAYMENT · ACCEPT_PAYMENT · PLACE_UNPAID_DIRECT · APPLY_VOUCHER

Module	Permissions
Customer Account (2)	CUSTOMER_ACCOUNT · SEND_ACCOUNT_BILL
Purchase Orders (9)	PURCHASE_ORDERS · PO_CREATE · PO_EDIT · PO_DELETE · PO_RECEIVE · PO_RETURN · PO_PAYMENT · PO_ADD_ITEM · PO_ADD_SUPPLIER
Suppliers (4)	SUPPLIERS · SUPPLIER_LEDGER · SUPPLIER_VIEW_LEDGER · Record Supplier Payment
Marketing (5)	MARKETING · MARKETING_CAMPAIGN · MARKETING_CAMPAIGN_LOG · MARKETING_CONTACTS · MARKETING_AUTO_SEND_LOG
Riders (9)	RIDERS · RIDER_VIEW · RIDER_CREATE · RIDER_EDIT · RIDER_DELETE · DUMMY_RIDER_VIEW · DUMMY_RIDER_CREATE · DUMMY_RIDER_EDIT · DUMMY_RIDER_DELETE
Other Screens (5)	KDS · EXPENSES · REPORTS · MY_TILL · PAYROLL
Security (1)	REQUIRE_PIN_FOR_ORDER_UPDATE (restriction: demand the order-update PIN before saving a reopened order)

Rules worth copying verbatim in behaviour

- Deleting a rider who already appears on past orders deactivates them instead, so delivery history is never orphaned
- PLACE_UNPAID_DIRECT does nothing unless the owner has also turned on Direct Unpaid Orders in settings, and reopened orders always go through the payment screen regardless
- APPLY_VOUCHER does not count against a cashier's discount limit, because the voucher value was fixed when the batch was created, and a voucher only works for the customer it was issued to
- Cashiers can create suppliers and inventory items inline while building a purchase order, but only with the matching permission

For the clone: model this as a `permissions` table of codes, a `role_presets` table mapping preset to codes, and a `user_permissions` join carrying the surface. Cache the granted set in the JWT at sign-in so the next-sign-in behaviour falls out naturally, and check it both in the API and in the UI.

Dashboard and the business day

Everything on the dashboard is scoped to a **business day that runs 6:01 am to 2:01 am**, not midnight to midnight. The top bar states that window explicitly alongside the live clock. Any clone that reports on calendar days will disagree with the restaurant's own numbers every night, so make the business-day window a configurable setting and derive every daily figure from it.

Top bar: current date and time, the business-day window, plan badge, a theme switch, a low-stock pill linking to items at or below minimum, an open-orders pill reading "22 open orders today · 80 from previous days", notifications, and the account menu showing the signed-in user and outlet name.

KPI row, each card scoped to the selected period:

Card	Definition
Total Products	Count of catalog products
Total Revenue	Completed orders only
Incl. Open Orders	Completed revenue plus the value still open, shown as "+ PKR x still open"
Total Orders	All orders, with cancelled called out separately
Total Customers	Registered customer count
Expenses	Expense total for the period

Charts

- Sales Trend: revenue from completed orders by hour
- Orders: order count by hour
- Products by Category: donut of catalog distribution
- Order Status Distribution: donut of statuses for the period
- Expenses: by category, with an explicit empty state

- Top Selling Products: ranked list with units sold and revenue per item

A period selector (Today and other ranges) and a Refresh button sit above the grid. Keeping revenue-completed and revenue-including-open as two separate headline numbers is a deliberate choice worth copying: in a restaurant with 80 orders still open from previous days, one number alone misleads.

Menu and catalog

The catalog is four levels deep: **Menu → Category → Product → Variant**, and each variant carries its own recipe. The reviewed account runs 155 products across 17 categories and 3 menus.

Menus group categories so the same restaurant can swap its offering. Each menu has a name, a created date, a category count and a *Show on website & app* toggle; a hidden menu keeps its categories and products out of the customer-facing site while staff can still sell them. The account uses this for Ramadan Iftar and Sehri menus alongside the default.

Categories sit inside a menu with a name, a subtitle and a product count. Categories and the products inside them are drag-and-drop sortable through an *Arrange* mode, and that order is what the POS and the website show. A *Favorites* set pins the most-used items to the front of the POS.

Products are created through a three-tab form.

Tab	Fields and behaviour
Basic Info	Product image (PNG/JPG/WEBP, max 5 MB), Product Name (required), Description, Base Price (required), Discount with a %/Rs switch, PCT Code for PRA e-IMS tax reporting, and a repeatable Variant row of Variant Name plus Price (for example Small, Medium, Large)
Ingredients	The recipe, set per variant. Each variant gets its own ingredient quantities, and the picker only offers units compatible with the chosen ingredient

Tab	Fields and behaviour
Order Inventory	Consumables charged by order type rather than by recipe — boxes, bags, fuel, supplies — deducted automatically at checkout. Set per order type and per variant, with a *Copy all order types* shortcut and per-order-type copy inside each card

Products show a price and an Active or Inactive state on their card. The catalog also offers *Download Menu*, *Import JSON* for bulk loading, a Display Settings panel and a Product Codes panel.

Deals are fixed-price combos: a deal name, a price, a customer-facing description and a list of component products with quantities. The account runs 10 active deals, priced from Rs 950 to Rs 2,790. Each deal can be favourited, viewed, edited, deleted, or added straight to an order.

Promotions are limited-time offers tracked by units sold and marked Sold Out when their cap is reached, so they carry a stock limit and a date window that deals do not.

Quick Add Menu is a short pinned list of fast movers such as roti, water and drinks that staff can fire onto a running order in one tap without reopening it. Items are picked from the full product list into that list.

What the clone needs

- Variant-level pricing and variant-level recipes, not product-level only
- A separate consumables mapping keyed on order type, applied at checkout
- Sort order persisted on categories and products, with a favourites flag
- Per-menu online visibility, independent of a product's active flag
- A discount field on the product itself that accepts either a percentage or an amount
- PCT code on the product for Pakistani tax reporting

Orders, order taking and tables

An order carries a serial number and a separate order code, a type, a source, a status, a payment status and a full audit trail of who touched it. The account runs around 240 orders a day.

Three order types — Walk-in, Takeaway, Delivery — each gated by its own permission, and each able to require a customer or not by setting.

Five sources — Counter (POS), Website, Mobile App, QR Table and the Online Orders bucket — so the same order list covers the till and every digital channel.

Statuses are Pending, Preparing, Ready, Dispatched, Completed and Cancelled, with *Open (not completed)* as a filter across the first four. Payment status runs separately: Paid, Pending, Unpaid, or settled to Account or Complimentary.

Orders Management lists order cards with serial, code, type, status, customer, rider, timestamp, payment method and status, and total. It filters by type, status, payment method, source, rider and date range; sorts by newest, oldest, serial ascending or descending, and amount; and totals orders, revenue, average order value and completed count. Bulk actions are Mark All Completed, Export CSV and Sync Complimentary.

The order detail opens as a side panel with three tabs.

Tab	Contents
Order Details	Customer name and phone, delivery rider and address, order items with variant, quantity, unit price and line total, payment summary (subtotal, total, method, status), and the order story
Cost Analysis	Revenue, COGS, total profit and margin percentage for that single order, itemised by ingredient with the quantity deducted, plus labour cost lines mapped to products
Payments	Payment status, order total, method, and every payment record against the order

The order story is the standout feature. Every order records who punched it, who completed it and who collected payment, then a timeline of changes: punched, reopened for editing, edited, completed, each stamped with user, role and time. An edit shows the item diff and the money diff, for example *+ Cold Drink (1.5 Ltr) × 1, amount due 980 → 1230*. A reopen-and-edit count sits in the header. For a cash business this is the control that makes staff theft visible, and it should be built into the order model from day one rather than bolted on.

Reopening is a first-class flow, not an edit. It needs REOPEN_ORDER, can require a PIN, and has its own two permissions for deleting a line and for decreasing a quantity, because those are the two moves that can hide money.

Pending Orders gets its own screen for orders left open across days, with Calendar and List views and tiles for today, yesterday, the last 30 days and all time. The reviewed account had 22 open today and 80 from previous days, with the oldest from February. A reminder setting puts a badge in the POS and admin headers, turns it red near closing time and warns at logout.

Tables carry a name, a location zone (Stairs, Roof 2, Reception Roof, Terrace), a capacity and a status of Available, Occupied, Reserved or Maintenance, with tiles for total tables, available, occupied and total capacity. Tables tie to the QR Table order source, so a diner scans and orders against a table.

Riders. Permanent riders carry contact details, vehicle information, a per-delivery charge, and assigned-versus-completed delivery counts. Temporary riders are throwaway records with a lifetime — one order, N orders, the rest of today, or N hours — each with its own permissions. A **Rider Vault** tracks company cash riders are holding, with Issue, Return, Spend and Adjust movements and a per-rider statement.

Kitchen display, printing and devices

ARFAPOS is not one app. The admin panel reviewed here is the back office; around it sit a desktop POS, a tablet app, a kitchen display, a rider app and a customer-facing website, and the permission badges make the split explicit — *Software* for the desktop till, *App* for the tablet, *Both* for a shared setting.

Surface	Who uses it	Notes
Admin panel (web)	Owner, managers	Everything in this spec
Desktop POS	Cashiers, supervisors	Takes payment, drives printers, works offline and queues orders
Tablet app	Order takers	Punches orders on the floor; prints through the desktop's print server
Kitchen Display	Kitchen	Its own login role, no till access
Rider app	Delivery riders	Assignment and delivery status
Website and mobile app	Customers	Menu, ordering, QR table, reviews

Kitchen Display. The KDS is reached by its own permission and can be the only thing a user has, which is how the account runs a dedicated Kitchen User login. Store-wide alert settings control it: a sound when a new Pending order arrives (only while the KDS page is open), an order timeout in minutes after which a still-Pending card turns red, and a one-shot sound when a card first turns red. Both switches are off by default and sync to the KDS on next login or refresh.

Printing. Printers are configured on the desktop POS behind the PRINTERS permission. Tablets do not configure anything: the admin panel stores the desktop computer's IP as a print server and every order-taker tablet connects to it automatically. Receipt branding is set centrally — a logo printed at the top, a QR code in the footer for a review link or menu URL, primary and secondary contact numbers under the store name, and the thermal paper width.

Offline behaviour is designed in rather than hoped for. The desktop POS keeps an Offline Orders queue of orders punched without a connection, with its own permission to view it, and POS-facing settings are described throughout as syncing on next connection, next login or refresh and being cached for offline use. The inventory safety net exists for exactly this reason: an order can complete while its stock deduction call is lost.

For the clone: make the API the single source of truth and give each surface a token with its own permission set. Orders need a client-generated idempotency key so an offline queue can replay safely, and the kitchen display should run on a websocket channel rather than polling.

Payments, charges, tax and the finance ledger

Every payment method is also a finance account with a running balance, so taking an order and booking the money are the same event. The reviewed account holds Rs 1.25 million across six active accounts.

Payment methods. Four system defaults ship with every account — Cash, EasyPaisa, JazzCash and Bank — and a catalogue of Pakistani wallets and banks can be added: uPaisa, SadaPay, NayaPay, HBL Pay, UBL Omni, MCB, Allied, NBP, Bank AL Habib, Soneri and Dubai Islamic. Adding one creates a matching finance account and syncs it to every active cashier. An account with transaction history cannot be deleted. Beyond these, an order can be settled to **Account** (customer credit) or marked **Complimentary** or **Unpaid**.

Finance Dashboard shows total balance across accounts, Today In and Today Out, and counts of the four movement sources — orders, expenses, supplier payments and transfers — then lists recent activity with the account balance after each line.

Charges. Service charge and delivery charge are each configured as a percentage or a fixed amount and pre-fill on the POS payment page and the delivery order page respectively; the cashier can change or remove either per order. Setting the value to zero or blank disables the default. Both sync to the POS on next connection.

Tax is wired to PRA e-IMS, Punjab Revenue Authority fiscal invoice filing, configured once per POS: a sandbox or production environment, a POS ID and access code from PRA registration, a bearer token held server-side and used only by the cashier desktop app, plus a counter name and branch name. Tax rates are linked to finance accounts — a cash payment rate (typically 16%) and a reduced card rate — and the cashier only sees a No Tax / 16% choice when a linked account is selected. Products carry a PCT code for the same filing.

Order payment behaviour is governed by settings rather than hard-coded:

- An order confirmation popup after each order, on by default; off means auto-redirect
- Direct Unpaid Orders, which skips the payment screen entirely for cashiers who also hold PLACE_UNPAID_DIRECT. Admins always qualify, and reopened orders always use the payment screen
- Which method the payment screen pre-selects. It is a pre-selection only; the cashier still presses Complete and may switch. Reopened orders always open on their original method
- Cashier Drawer, which routes payments into each cashier's own till account instead of the admin accounts

Double-entry accounting sits underneath. A system chart of accounts ships pre-numbered — 1000 Current Assets, 1010 Cash in Hand, 1020 Cash at Bank, 1030 Petty Cash, 1100 Accounts Receivable, 1200 Inventory, 1300 Prepaid Expenses, 1500 Fixed Assets through 1590 Accumulated Depreciation, 2000 Current Liabilities and so on — each account carrying a code, a type, a normal side and an opening balance, with System accounts protected. On top of it: Journal Voucher for manual entries, Combined Ledger, Fixed Assets, Liabilities, Equity, Trial Balance and Balance Sheet.

For the clone: treat a payment as a ledger entry against an account, never a column on the order. Orders can be part-paid, settled to credit, or complimentary, and the order's payment status is derived from its payments. Keep the chart of accounts seeded and flagged as system rows so users can extend but not break it.

Inventory and recipes

Stock is held per section, moved only through numbered documents, and deducted automatically when a sale completes. The account carries 184 items, of which 164 hold stock worth Rs 1.6 million, across two sections, Kitchen and Warehouse.

Items. Each item has a name, an auto-generated SKU (first letters of the name, a category code, four digits — for example `ADR-GE-9350`), a purchase category such as ST KARYANA or General Local Purchase, a base unit, a minimum stock level and an active flag. Quantities are read-only on the item screen; the screen says plainly that stock changes only through Stock Actions and sales. Items can be exported and imported by CSV. Negative stock is permitted and displayed as **short**, because a sale is never blocked over a stock figure.

Units and conversions are first-class. Twelve units (piece, dozen, gram, kilogram, millilitre, litre, box, bottle, packet and custom ones such as Pet12 and Pet6) and ten stored conversion factors, each direction stored separately, for example 1 kg = 1000 g and 1 pt12 = 12 btl. When a recipe's unit differs from the item's unit the system converts automatically, and the ingredient picker only offers compatible units.

Display units are a separate setting from storage. Weights, volumes and counted items each read as Automatic or a fixed unit everywhere in the system; 25,000 g reads as 25 kg while nothing stored changes.

Recipes exist at two levels.

1. Product recipes, set per variant inside the product form, which deduct on sale
2. Standalone recipes, which are prep batches with a yield — Sauce yields 2 kg at Rs 1,102.59, or Rs 551.29 per kg — a source section, and their own ingredient lines. These feed Production stock actions

Stock Actions is a document register. Every movement is a numbered document with a type, a status and a posting date.

Field	Values
Type	Goods Receipt · PO Receipt · Transfer · Waste · Adjustment · Stock Count · Opening Balance · Production
Status	Draft · Awaiting approval · In transit · Posted · Voided

Field	Values
Carried	Number (GRN-000023), linked PO number, date, sections, line count, total value, created-by, timestamp

Inventory Settings holds the rules every surface obeys, and these are the behaviours most worth copying exactly:

- **Day lock.** No stock movement may be dated on or before the locked date, so a checked and closed day cannot change. Sales are exempt: an order on a locked day still deducts, and a cancellation still returns stock
- **Stock counting on/off.** A global switch that closes counting entirely; posted counts are untouched and every other action carries on
- **Allow negative on manual movements.** When off, a transfer, return or production that would go below zero is refused with the quantity actually on hand. Sales and waste are always allowed below zero
- **Return stock on cancellation.** When on, cancelling an order that already deducted puts the ingredients back, from every app. When off, the expectation is that staff record waste instead
- **Safety net.** The till deducts once, at completion. If that call is lost, a job sweeps every ten to fifteen minutes, finds completed orders whose stock never moved and deducts them **dated to the day the order was sold**, never twice

Demand answers what to restock. Filtered by section and category and bucketed as Low stock, Critical, Out of stock, All needing stock or Everything in use, it checks each item's stock against its minimum and recent usage, subtracts what is already on order, and splits the answer into what to transfer and what to buy.

Deduction Health is a diagnostics screen with no equivalent in most POS products and is worth building early. It reports whether the safety net is running, lets you run it on demand, checks any single order and lists every stock movement written for it, and buckets problem orders — for example completed orders whose deduction never ran — with a Retry action. In the reviewed account it flagged 75 of 243 orders as needing a look.

For the clone

- Stock lives in a `stock_movements` ledger, never as a mutable quantity column. Current stock is the sum, cached per item and section

- Deduction is an idempotent background job keyed on the order, so a retry can never double-deduct
- Movements carry a posting date separate from their created-at, so a safety-net deduction books to the sale's day
- Keep quantities in the item's base unit at full precision and convert only at the edges

Purchasing, suppliers and expenses

Suppliers run on a real running-balance ledger, not a list of invoices. Lifetime figures in the reviewed account: Rs 5.34 million purchased, Rs 3.57 million paid, Rs 479,186 outstanding across 18 of 23 suppliers.

Suppliers carry a name, a contact person, phone, an address or area, a balance with a direction label (**To Pay**, **To Receive**, **Settled**), a count of distinct items supplied and a count of purchases. Import and export by CSV. Cashiers with the right permission can create a supplier inline while building a purchase order.

Purchase orders list PO Number, Date, Supplier, Items, Total Amount, Status and Payment, with KPI tiles for total POs, drafts, received and total value, filters on status, supplier and date, and a PDF export. The lifecycle is create → receive stock (which writes a GRN stock action) → pay, each gated by its own permission, with purchase returns as a separate screen.

Supplier Ledger is the accounting view: a transaction ledger of Dr and Cr lines with a running balance, each line typed as Purchase, Payment or Adjustment and linked to its PO, plus an Aging Summary tab. Headline tiles show opening balance, total purchases, total payments and net balance owed. Manual adjustments are entered here.

Supplier Insights ranks suppliers by purchased, paid, outstanding or name, showing PO count and last purchase date per supplier.

Purchase Price History is a data-quality tool and the most interesting screen in this module. It sets what you actually paid per item against the average cost sales are booked at, over a chosen window, and flags items that need a look with a plain verdict:

Column	Meaning
Bought	Quantity and number of purchases in the window
Spent	Total spend on that item

Column	Meaning
Paid on average	Spend divided by quantity, in the item's unit
Last price	Most recent purchase price and its date
Average cost	The cost sales are currently booked at
Difference	Percentage gap between the two
Verdict	For example *Wrong unit on a purchase*, *Valued below what you pay*

In the reviewed account 19 of 111 purchased items were flagged, almost all of them unit-entry mistakes. For the clone this is worth building, because recipe costing is only as good as the unit on the purchase line.

Expenses are separate from purchases. Each expense has a category and subcategory, a payment method and a source (entered by a cashier at the till or by an admin). The screen filters by all of those with a date range, shows total, average, count and category count, and exports to PDF and CSV. Categories are managed from the same screen.

Staff, attendance and payroll

Attendance runs off a fingerprint kiosk and feeds payroll directly: the scanner counts the days and payroll prices them. This is the most rule-heavy module in the system and deserves its own sprint.

Attendance screens: Today (live check-in and out), Sessions (full log), Scan Log (every tap on the reader), Ledger (month view per employee), Enrolled (manage and revoke fingerprints), Reports, Leave (paid and unpaid), Holidays (Eid, Muharram, closures), Employee Rules (per-person shift and fines) and Defaults (business-wide rules). The Today view counts total scans, currently in, checked out, not checked in and late, against the business date.

The rules engine. A master switch decides whether rules apply at all; with it off every scan simply records as Present. Business hours come from Store settings (6:01 am to 2:01 am here) and act as the last fallback for a shift start. Per-employee rules override the business defaults.

Rule	Behaviour
Thresholds	Late after, Half day after, Absent after — each in minutes from that employee's shift start, judged on the first scan. A blank field skips that rule
Status timing	Status is decided at check-in only; checkout never changes it
Overtime	Off by default. Measured from shift end and only when the employee actually clocks out. A day with no check-out earns nothing and is flagged as a missing punch rather than guessed at
Weekly offs	Selected days are never counted absent
No-scan day	Optionally counts as absent. Holidays, weekly offs and approved leave are always excluded
Free allowances	First N lates, half days and absents per month are forgiven before any charge
Penalties	Per status, one of Fine only, Lose the day's pay, Both, or Not charged, plus a flat fine amount. A lost day is valued by the payroll divisor; half-day and absent deductions apply automatically, so a fine on top is optional
Fine cap	Optional monthly cap per employee. Caps fines only — lost-day deductions are never capped
Checkout gap	Cannot check out within N minutes of check-in. A second scan inside the gap shows "Already Marked". Manual checkout by an admin or cashier is always allowed, and anyone who never checks out is auto-closed at the end of the business day

One implementation detail worth copying: changing the rules does not rewrite history. Rules apply to new and updated records, and rows already saved keep their stored status until they are scanned or edited again.

Payroll. The overview tracks total employees, pending advances, whether this month's payroll has been generated, and outstanding advances approved but not fully repaid. Its screens are Employees, Advances, Run Payroll, Ledger, Reports and Settings, with a **Sync**

to Expenses action that pushes payroll cost into the expense ledger.

Payroll settings hold two decisions:

1. **Salary calculation method.** Actual Calendar Days divides monthly salary by the real days in the month, so the daily rate moves between 28 and 31 day months; Fixed Divisor always divides by the same number, commonly 26. The screen shows the difference worked out on a Rs 30,000 salary: Rs 1,071.43 deducted for a February day against Rs 967.74 for a January day
2. **Require fingerprint to close shift.** When on, a cashier must scan at the attendance reader before the POS lets them close their shift, which forces an accurate departure time

For the clone: keep attendance statuses as stored values on each day's record rather than computing them on read, since the rules change over time and history must stay stable. Price the days in a separate payroll run that snapshots the divisor, the fines and the allowances it used.

Customers, credit, loyalty and feedback

Customers are ledger accounts, not just contact records. Each carries a **Khata No.**, the running credit-account number local restaurants work with, and a balance that reads Settled or To Receive. The account holds 891 customers.

Customer records: Khata No., name, phone and email, address, source (In-store or Online), balance, status, plus grouping. Lifecycle states are Active, Pending approval and Deactivated, so online sign-ups can be held for approval. Customers can be imported by CSV and organised into groups.

Credit accounts. Orders can be settled against a customer's account rather than cash, which is why Account appears as a payment method and AR Aging as a report. Supporting screens are Customer Ledger, Customer Payments, Customer Insights and AR Aging.

Account notifications over WhatsApp. A dedicated screen controls statements and receipts sent from the cashier desktop app: a master toggle that disables every send button everywhere, auto-send on account payment, optional receipt image, optional store logo on images, plus Templates, Manual Send, Bulk Send and Send Logs. Templates are built from variables including `{customer_name}`, `{order_number}`, `{order_total}`, `{order_items}`, `{discount}`, `{service_charge}`, `{loyalty_points}`, `{previous_balance}`, `{new_balance}` and `{business_name}`.

Three discount mechanisms exist and they do not overlap.

Mechanism	How it works
Loyalty points	Rules issue points, customers accumulate them, redemption options convert them back. Tracked by points issued and redeemed
Regular discounts	Recency tiers: *if their customer's last order was within N days, take X off*. The POS applies the best match automatically, and the smallest day window wins, so a customer who ordered yesterday gets the 1-day reward rather than the 7-day one. Customers who have never ordered get nothing. Exclusions work per tier, plus a global blocked list
Vouchers	Code batches with a value, a minimum spend and an expiry. **Personal** codes are generated per customer and sent from the POS campaign screen, redeemable only by that customer. A **public** code is a single code posted on social media, usable on any order until it expires or hits its usage limit

Because a voucher's value was fixed when the batch was created, applying one does not count against the cashier's own discount limit. That rule matters: it is what lets an owner hand out vouchers without widening what cashiers can discount on their own.

Reviews come in from the tablet and from the website, each tied to an order and its amount, with a 1-to-5 rating shown as a word, an optional comment, order type and date. The screen reports total reviews, average rating and the 5-star and 1-star counts, and filters by rating, source, type and a Pending (Unrated) state.

Settings and multi-outlet

Settings are grouped into five areas, and almost every POS behaviour described elsewhere in this spec is switchable here rather than hard-coded. That is the right instinct to copy: restaurants differ, and the owner should not need a developer.

Group	Screens
Store	Store Profile · Store Management · Business Hours
Point of Sale	Order Taker · Customer Required · Customer Fields · Order Behavior · POS Notifications · KDS Alerts · Barcode Scanning · Pending Order Reminder · Receipt Branding · Print Server
Charges & Tax	Service Charge · Delivery Charge · PRA e-IMS
Inventory	Sections · Stock Alerts · Reset Stock
Security & Cash	Change Password · Signed-In Devices · Expense & Report PIN · Cashier PIN · Cashier Drawer

Store Profile holds the business name, primary and secondary phone and address, all of which print on the receipt. **Business Hours** defines the business day — 6:01 am to 2:01 am in this account — and that window is read by the dashboard, every daily report and the attendance rules engine, which treats it as the last fallback for a shift start.

Customer Required makes customer selection mandatory per order type, independently for walk-in, takeaway and delivery, enforced by the cashier apps at order placement and cached for offline use. **Customer Fields** controls which details are asked for.

Security and cash is where the money controls live: separate PINs for expenses and reports and for cashier actions, a signed-in devices list, and the Cashier Drawer switch that decides whether payments land in each cashier's own till or in the admin accounts.

Multi-outlet is handled by **Store Management**, which links other store accounts so several brands or branches can be managed from one dashboard. The website config has its own Branches screen. The reviewed account has no linked stores, so branch behaviour could not be confirmed.

The storefront is configured from Website Config, a full site builder: Store Identity (name, URL slug, description, theme colour with presets, logo, cover image, favicon), Pages, Menu Slider, Documents, Footer, Availability, Customer Sign-ups, Location, Branches, Delivery, Pricing, Contact, Social Media, WhatsApp, Instagram, SEO, Mobile App and System.

Other standalone settings screens: WhatsApp, Notifications, Plan (subscription tier, Business here) and **Audit Logs**.

Business Hours is two fields, start and end, and the end may fall on the next day. The screen states the reason plainly: an order at 1:00 am belongs to the previous day's report, so midnight never splits an operational period.

Inventory Sections are extensible. Kitchen and Warehouse are system sections that cannot be removed, Warehouse is flagged **COGS Primary**, and custom areas such as BBQ, Bar or a station can be added. Each carries a slug; a new section appears in the Inventory sidebar and in the catalog's source picker, and items assigned to it must have their location set to that slug.

Audit Logs records every create, update and delete made through the admin panel and is append-only, with nothing erasable. It filters by date range, by actor, by action — create, update, delete, bulk update, bulk delete, cancel, receive, status change, ledger debit, ledger credit — and by entity, with more than thirty entity types tracked including orders, products, menus, deals, customers, suppliers, purchase orders, stock history, order stock deductions, payment accounts, both ledgers, cashiers, cashier session settlements, payroll employees, website configuration and WhatsApp settings. It also has a Search Insights tab and a CSV export. For the clone this is a single polymorphic `audit_logs` table written from a model concern, and it is cheap to add in phase 1 but painful to backfill later.

For the clone: hold settings as a typed key-value store scoped to the outlet, with a seeded default per key and a version stamp each surface can compare against, so a POS knows when to re-sync. Group them exactly as above — it is a good information architecture and users will already know it.

Report catalogue

Twenty-seven report screens in all, nineteen under Reports and eight under Inventory Reports. Most take a date range and export to CSV or PDF.

Report	What it answers
Business Overview	Headline sales and order metrics for the period
Grand Report	Sales split by every payment method with counts and share, Unpaid and Complimentary as a share of sales, plus payments received against customer accounts
Sales	Sales detail over the period

Report	What it answers
Sales Calendar	Day-by-day sales in a calendar grid
Financial	Revenue against costs and expenses
Products & Deals	Item and deal performance
Category Report	Sales by menu category
Order Takers	Orders punched per order taker
Cashiers	Cashier activity
Cashier Till	Who rang orders up against who holds the money
Customers	Customer activity and spend
Customer Ledger	Credit account movements
AR Aging	Outstanding customer balances by age
Operations	Operational throughput
Cancelled & Modified Orders	Every cancellation and edit, the theft-and-error report
Delivery Riders	Deliveries and charges per rider
Vouchers	Voucher issue and redemption
Loyalty Points	Points issued and redeemed
Regular Discounts	Automatic tier discounts given
Daily Stock Sheet	One day's stock in, out and closing per item
Stock Ledger	Every movement per item
Consumption	What was used over the period
Waste & Adjustments	Waste and manual corrections
Actual vs Theoretical	Counted stock against what recipes say should remain
Valuation & COGS	Stock value and cost of goods sold
Deduction Health	Whether sales are deducting stock correctly

Report	What it answers
Purchase Price History	Paid price against booked average cost, with verdicts

Cashier Till deserves attention because it encodes a distinction most POS reports get wrong. Orders taken and money collected are different measures and deliberately do not reconcile: an order rung up by one cashier and paid to another counts once for each. Only Collected is a figure someone is accountable for. It also carries an **In tills now** balance that runs all-time until a Day-End Settlement clears it, so an unsettled till shows more than the period's collections. Build the settlement record first; the report is meaningless without it.

Two AI reports sit alongside: **AI Pricing Analysis**, which reads a period's sales and suggests underpriced items and revenue opportunities, and **AI Stock Forecast**. Both are advisory layers over the same data, which makes them a sensible phase-two addition for the clone rather than a core dependency — a scheduled job that sends aggregates to an LLM and stores the suggestions.

Data model

Every table carries `outlet_id`, and three subsystems are append-only ledgers rather than mutable balances: stock, money and the order audit trail. Get those three right and the rest is ordinary CRUD.

Identity and access

Table	Key columns
`outlets`	name, slug, timezone, business_day_start, business_day_end, currency
`users`	outlet_id, name, email, phone, password_digest, user_type (admin/cashier/order_taker/rider), role, active
`permissions`	code, module, label, description, surface (software/app/both), restriction
`role_presets` / `role_preset_permissions`	preset name, permission codes
`user_permissions`	user_id, permission_code, granted_at

Table	Key columns
`user_sessions`	user_id, device, signed_in_at, signed_out_at, active

Catalog

Table	Key columns
`menus`	name, visible_online, position
`categories`	menu_id, name, subtitle, position
`products`	category_id, name, description, image, base_price, discount_value, discount_type, pct_code, active, favorite, position
`product_variants`	product_id, name, price, position
`variant_ingredients`	variant_id, inventory_item_id, quantity, unit_id
`order_type_consumables`	product_id, variant_id, order_type, inventory_item_id, quantity, unit_id
`deals` / `deal_items`	name, price, description, active / product_id, variant_id, quantity
`promotions`	name, price, starts_at, ends_at, unit_cap, units_sold
`quick_add_items`	product_id, position

Inventory

Table	Key columns
`sections`	name (Kitchen, Warehouse)
`units` / `unit_conversions`	name, abbreviation, kind, is_default / from_unit_id, to_unit_id, factor
`inventory_items`	name, sku, purchase_category, base_unit_id, minimum_quantity, active

Table	Key columns
`recipes` / `recipe_lines`	name, description, yield_quantity, yield_unit_id, section_id / inventory_item_id, quantity, unit_id
`stock_documents`	number, doc_type, status, posting_date, from_section_id, to_section_id, purchase_order_id, total_value, created_by
`stock_document_lines`	document_id, inventory_item_id, quantity, unit_id, unit_cost
`stock_movements`	inventory_item_id, section_id, quantity_base, posting_date, source_type, source_id, unit_cost
`stock_balances`	inventory_item_id, section_id, quantity_base, average_cost (a cache, rebuildable from movements)

Orders and money

Table	Key columns
`orders`	serial, code, order_type, source, status, customer_id, table_id, rider_id, subtotal, service_charge, delivery_charge, discount, tax, total, payment_status, business_date, idempotency_key, punched_by, completed_by
`order_items`	order_id, product_id, variant_id, deal_id, quantity, unit_price, line_total
`order_events`	order_id, event_type (punched/reopened/edited/completed/cancelled), user_id, occurred_at, diff (jsonb), amount_before, amount_after
`order_payments`	order_id, payment_account_id, method, amount, received_by, received_at
`payment_accounts`	name, method_key, is_system, balance_cache, cashier_id

Table	Key columns
`account_transactions`	payment_account_id, direction, amount, source_type, source_id, balance_after
`chart_of_accounts`	code, name, account_type, normal_side, is_system, opening_balance
`journal_entries` / `journal_lines`	number, date, memo / account_id, debit, credit
`tables` / `table_zones`	name, zone, capacity, status

Parties

Table	Key columns
`customers`	khata_no, name, phone, email, address, source, status, group_id, balance_cache
`customer_ledger_entries`	customer_id, direction, amount, source_type, source_id, balance_after
`suppliers`	name, contact_person, phone, address, balance_cache
`supplier_ledger_entries`	supplier_id, direction, amount, entry_type, purchase_order_id, balance_after
`purchase_orders` / `purchase_order_lines`	number, supplier_id, status, payment_status, total / inventory_item_id, quantity, unit_id, unit_price
`riders`	name, phone, address, vehicle, delivery_charge, temporary, lifetime_rule, expires_at, active
`rider_vault_entries`	rider_id, entry_type (issue/return/spend/adjust), amount, balance_after

People and time

Table	Key columns
`employees`	user_id, name, base_salary, shift_start, weekly_offs, active
`attendance_scans`	employee_id, scanned_at, device, direction
`attendance_days`	employee_id, business_date, check_in, check_out, status, late_minutes, overtime_minutes, fine_amount, rules_snapshot (jsonb)
`attendance_rules`	scope (business/employee), employee_id, thresholds, allowances, penalties, fine_cap, checkout_gap
`holidays` / `leaves`	date, name / employee_id, from, to, paid, approved_by
`payroll_runs` / `payroll_lines`	period, divisor_method, divisor, generated_at / employee_id, gross, deductions, fines, advances, net
`advances`	employee_id, amount, approved_by, repaid_amount, status

Marketing, feedback and trail

Table	Key columns
`loyalty_rules` / `loyalty_points` / `redemptions`	earn rule, points balance, redemption option
`regular_discount_tiers`	within_days, discount_type, discount_value, exclusions, active
`voucher_batches` / `voucher_codes`	kind (personal/public), value, minimum_spend, expires_at, usage_limit / code, customer_id, used_at, order_id
`reviews`	order_id, customer_id, rating, comment, source, created_at

Table	Key columns
`expenses` / `expense_categories`	category_id, subcategory, amount, payment_account_id, entered_by, business_date
`settings`	outlet_id, key, value (jsonb), version
`audit_logs`	user_id, action, entity_type, entity_id, changes (jsonb), ip, created_at

Rules the schema has to enforce

- Stock in the item's base unit at full decimal precision; convert only when reading or writing at the edges
- `orders.business_date` computed from the outlet's business-day window at insert, never derived at read time
- A unique index on `orders.idempotency_key` so an offline replay cannot duplicate an order
- `stock_movements` unique on (source_type, source_id, inventory_item_id) for sale deductions, which is what makes the safety-net job safe to re-run
- Balance caches on customers, suppliers, accounts and stock are derived values with a rebuild task, never the source of truth

Rails API design

One API serves every surface. Namespace it `/api/v1`, authenticate with JWT, and carry the user's permission codes in the token so each surface gets exactly the access its role allows.

Auth and context

- `POST /auth/login` returns a JWT holding user id, outlet id, user type and the granted permission codes, plus a refresh token
- Permissions are checked in a Pundit policy per action, mapped to the same codes the admin UI shows, so the code in the policy is the code the owner ticked
- Because the token carries the permission set, a change takes effect at next sign-in, matching the behaviour the admin screen promises
- Every request resolves the outlet from the token and scopes through a default scope; no endpoint accepts an outlet id from the client

Endpoint groups

Group	Endpoints
Catalog	`menus`, `categories`, `products`, `products/:id/variants`, `deals`, `promotions`, `quick_add_items`, plus `catalog/sync` returning the whole menu with an ETag for POS caching
Orders	`orders` (index with the full filter set), `orders/:id`, `POST orders`, `PATCH orders/:id/reopen`, `/complete`, `/cancel`, `/items`, `orders/:id/events`, `orders/:id/payments`
Tables	`tables`, `PATCH tables/:id/status`
Inventory	`inventory_items`, `stock_balances`, `stock_documents` with `/post` and `/void`, `recipes`, `units`, `unit_conversions`, `demand`
Purchasing	`suppliers`, `purchase_orders` with `/receive`, `/return`, `/pay`, `supplier_ledger`, `supplier_payments`
Customers	`customers`, `customer_ledger`, `customer_payments`, `customer_groups`, `ar_aging`
Money	`payment_accounts`, `account_transactions`, `expenses`, `journal_entries`, `chart_of_accounts`, `trial_balance`, `balance_sheet`
People	`cashiers`, `order_takers`, `riders`, `rider_vault`, `employees`, `attendance/scans`, `attendance/days`, `attendance/rules`, `payroll_runs`, `advances`
Marketing	`loyalty_rules`, `regular_discount_tiers`, `voucher_batches`, `voucher_codes/:code/validate`, `reviews`
Reports	One endpoint per report, each taking `from`, `to` and its own filters, returning rows plus totals

Group	Endpoints
Settings	`settings` (grouped read, patch by key), `settings/version` for POS sync checks

Order creation is the endpoint that matters most. `POST /orders` accepts an `idempotency_key` generated by the client, so an offline till replaying its queue cannot create duplicates. Creating an order runs in a transaction: write the order and its items, compute charges and tax, apply the regular-discount tier if a customer is attached, write payments and account transactions, then enqueue the stock deduction rather than doing it inline, so a slow deduction never blocks the till.

Background jobs

Job	Purpose
`DeductStockJob`	Writes stock movements for a completed order; idempotent on the order
`StockSafetyNetJob`	Every 10 minutes, finds completed orders with no movements and deducts them, posted to the sale's business date
`RecalculateBalancesJob`	Rebuilds cached balances for items, customers, suppliers and accounts
`AutoCloseAttendanceJob`	Closes unchecked-out attendance at the end of the business day
`PayrollRunJob`	Generates a payroll run, snapshotting the divisor, fines and allowances
`AiInsightsJob`	Sends period aggregates for pricing and stock-forecast suggestions

Realtime. ActionCable channels for the kitchen display (new and changed orders), the orders list, and the table map. The KDS timeout colouring is computed client-side from the order's created-at and the store's timeout setting, so no server tick is needed.

Next.js frontend plan

Three frontends share one component library and one API client: the admin panel, the POS, and the customer storefront. Build the admin panel first, because it is the only one you need in order to put real data behind the other two.

Apps

App	Route root	Notes
Admin	`/admin`	The 80 screens in this spec. Desktop-first, dense tables
POS	`/pos`	Touch-first order taking, payment, kitchen display, till. Installable PWA with an offline queue
Storefront	`/`	Public menu, ordering, QR table, reviews. Server-rendered for SEO

Admin route structure mirrors the module groups so URLs stay guessable: `/admin/products/*`, `/admin/inventory/*`, `/admin/purchasing/*`, `/admin/suppliers/*`, `/admin/customers/*`, `/admin/staff/*`, `/admin/finance/*`, `/admin/payroll/*`, `/admin/reports/*`, `/admin/riders/*`, `/admin/settings/*`.

State and data

- TanStack Query for all server state, with the query key carrying the filter set so date-range changes refetch cleanly
- Zustand for the POS cart only; everything else stays server state
- Zod schemas shared between form validation and API response parsing
- The permission set from the JWT held in a context, with a `<Can code="REOPEN_ORDER">` wrapper and a `usePermission` hook, so the UI hides what the API would refuse

Patterns that repeat across almost every screen and should be built once, early:

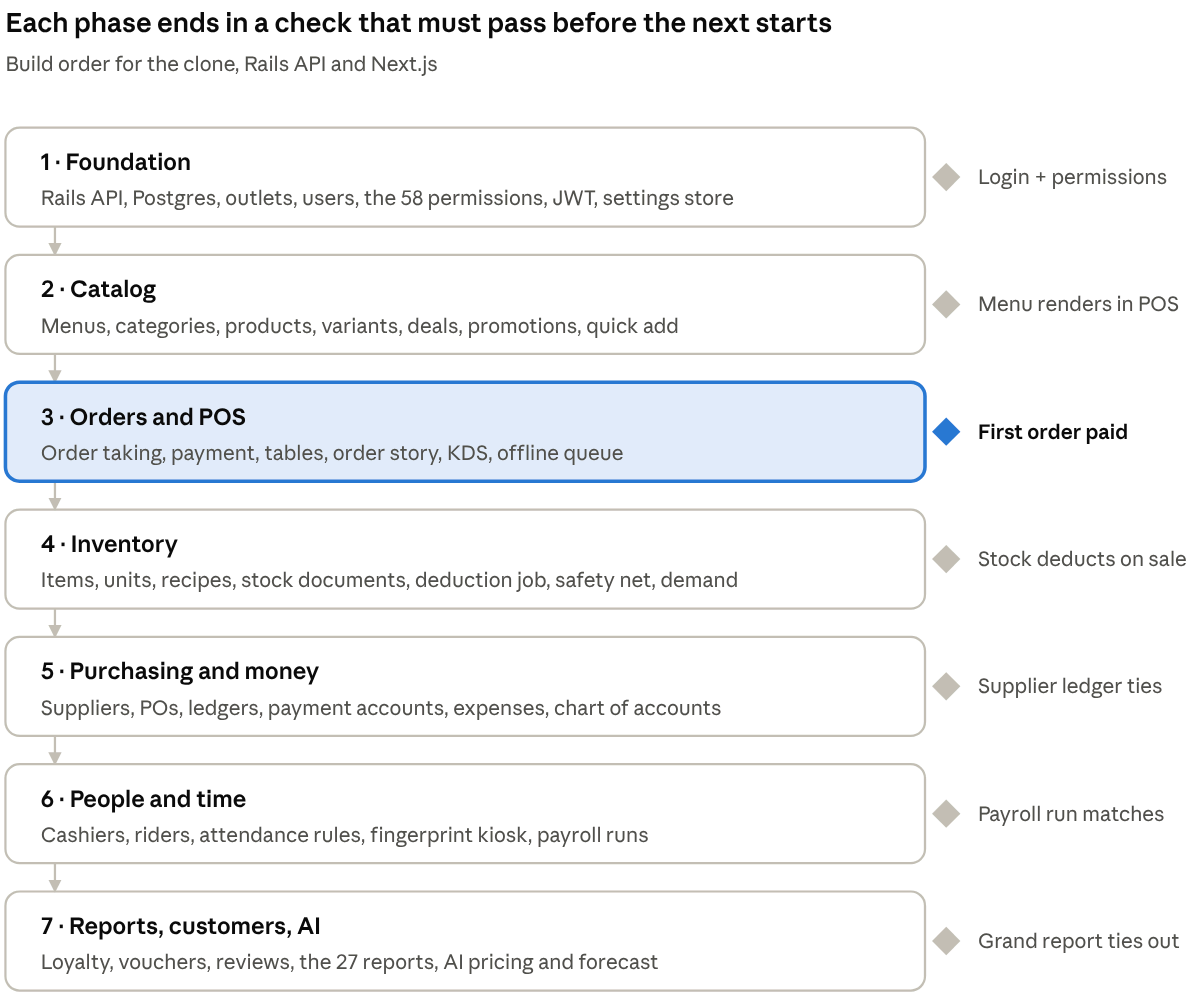
1. A filter bar: date range, a handful of dropdowns, search, sort, with state held in the URL so a filtered view is shareable
2. A KPI tile row above the table
3. A data table with server-side pagination, column sorting and CSV or PDF export
4. A right-hand detail drawer with tabs, as the order drawer uses

5. A multi-step form modal with a section rail, as the product form uses
6. An empty state with a primary action

POS specifics. The POS is the hard part. It needs the whole catalog cached in IndexedDB with an ETag check on launch, an order queue that survives a refresh, optimistic order creation keyed on a client-generated UUID, and a clear offline banner. Keep the payment screen a separate route so the Direct Unpaid Orders setting can skip it cleanly.

Charts. The dashboard and reports need hourly sales lines, category donuts and status breakdowns. Pick one charting library and one palette at the start and apply it everywhere, so the reports read as one product rather than twenty.

Build phases



build roadmap · 7 phases, each with its gate

Phase 3 is where the product becomes usable: a cashier can take an order and take money for it. Everything before it exists to make that possible, everything after it makes the numbers trustworthy.

Phase 1 — Foundation. Rails API skeleton, PostgreSQL, Devise with JWT, outlets, users and the four staff types, the permission table seeded with all 58 codes and the three presets, the settings key-value store with seeded defaults, the business-day helper, and the Next.js admin shell with the sidebar, filter bar, KPI row, data table and detail drawer built as reusable components.

Phase 2 — Catalog. Menus with online visibility, categories with sort order, products with the three-tab form, variants, the favourites flag, deals, promotions, quick-add items, image upload, and JSON import and menu export.

Phase 3 — Orders and POS. Order creation with an idempotency key, the three order types and five sources, the status machine, order items with variants and deals, charges and discounts, payments against accounts, the order story audit trail, reopen with its own permissions and PIN, tables with zones and statuses, the kitchen display on a websocket, and the POS PWA with an offline queue.

Phase 4 — Inventory. Items with SKUs and minimums, sections, units and conversions, display-unit settings, variant recipes, order-type consumables, standalone recipes with yields, the stock document register with its eight types and five statuses, the movement ledger, the deduction job, the safety net and Deduction Health, day lock, and Demand.

Phase 5 — Purchasing and money. Suppliers, purchase orders with receive, return and pay, supplier ledger with aging, purchase price history with its verdicts, customers with khata numbers and credit, customer ledger and AR aging, payment accounts and transactions, expenses with categories, and the seeded chart of accounts with journal vouchers, trial balance and balance sheet.

Phase 6 — People and time. Cashier sessions and tills with day-end settlement, order takers, permanent and temporary riders, the rider vault, employees, the fingerprint kiosk integration, the full attendance rules engine, leave and holidays, and payroll runs with advances and expense sync.

Phase 7 — Reports, customers and AI. The 27 report screens with shared filter and export components, loyalty rules and points, regular discount tiers, voucher batches, reviews, WhatsApp account notifications, the customer storefront, audit logs, and finally the two AI advisory jobs.

A realistic solo pace is two to three weeks per phase with Claude Code doing the bulk of the writing, so roughly four to five months to feature parity. Phases 1 to 3 alone give a working POS you could run a restaurant on.

The build prompt

Use this in two steps. Paste the project brief once to set up the repo and `CLAUDE.md`, then paste one phase prompt per phase. Keep this document in the repo as `docs/SPEC.md` so Claude Code can read the detail rather than carry it in the prompt.

Step 1 — project brief

```
You are building a restaurant management and point-of-sale platform from
scratch.
I am a senior full-stack engineer. Write production code, not a demo.

## Stack (fixed)
- Backend: Ruby on Rails 8, API-only, PostgreSQL 16
- Frontend: Next.js 15 App Router, React 19, TypeScript, Tailwind
- Jobs: Sidekiq + Redis. Realtime: ActionCable
- Auth: Devise + devise-jwt. Authorization: Pundit
- Tests: RSpec + FactoryBot (backend), Vitest + Testing Library (frontend)
- Monorepo: /api (Rails), /web (Next.js), /docs

## Read first
`docs/SPEC.md` is the full functional specification. Treat it as the
requirements document. When something is not in it, ask me rather than
invent.

## Non-negotiable architecture rules
1. Every table has outlet_id. Every query is scoped to the outlet from the
JWT.
   No endpoint accepts an outlet id from the client.
2. Stock is an append-only `stock_movements` ledger. Never a mutable quantity
   column. Current stock is a cached sum with a rebuild task.
3. Money is an append-only ledger too: `account_transactions`,
   `customer_ledger_entries`, `supplier_ledger_entries`. Balances are caches.
4. Orders carry a client-generated `idempotency_key` with a unique index, so
an
   offline till can replay its queue without creating duplicates.
5. Stock deduction runs as a background job, idempotent on the order, with a
   unique index on (source_type, source_id, inventory_item_id). A sweeper job
```

```
   re-runs missed deductions, posted to the order's business date.
6. The business day is a configurable window (default 06:01 to 02:01), not
the
   calendar day. `orders.business_date` is computed at insert.
7. Permissions are 58 string codes. They live in the JWT, are checked by a
   Pundit policy per action, and gate the UI through a <Can> component.
   Changing a permission takes effect at next sign-in.
8. Settings are a typed key-value store per outlet with seeded defaults and a
   version stamp the POS compares against to re-sync.
9. Quantities are stored in the item's base unit at full decimal precision.
   Convert only at the edges. Money is a cents integer, never a float.
10. Soft-delete anything referenced by history. A rider who appears on past
    orders is deactivated, never destroyed.

## How I want you to work
- One phase at a time. At the start of a phase, write a short plan and a task
  list, then implement it.
- Migrations and model specs first, then the API, then the UI.
- Every endpoint gets a request spec covering the happy path, the permission
  denial and one edge case.
- Run the tests and the linter before you tell me a phase is done.
- Commit per logical unit with a clear message. Do not commit secrets.
- Keep a running `docs/DECISIONS.md` of anything you chose that the spec
  did not settle.

## This product is original work
Build from the behaviour described in the spec. Do not copy any third-party
source code, templates, stylesheets, icon sets or brand assets. The name,
visual design and all copy are my own.

Start with Phase 1 only. Confirm the plan with me before writing code.
```

Step 2 — one prompt per phase. Use the same shape each time:

```
Phase 3 of the build: Orders and POS.

Read docs/SPEC.md sections "Orders, order taking and tables",
"Payments, charges, tax and the finance ledger" and
"Kitchen display, printing and devices".

Deliver:
- orders, order_items, order_events, order_payments tables and models
- POST /api/v1/orders with idempotency_key, in one transaction: order +
```

```
items,
  charges, discount tier if a customer is attached, payments, account
  transactions, then enqueue DeductStockJob
- Status machine: pending, preparing, ready, dispatched, completed, cancelled
- Reopen flow: REOPEN_ORDER, optional PIN, DELETE_ORDER_ITEM and
  MODIFY_REOPEN_DECREASE_QTY as separate permissions
- order_events writes a row for punched, reopened, edited, completed,
  cancelled, each with user, time and a jsonb diff of items and amounts
- Orders index with every filter in the spec, plus KPI totals
- Order detail drawer with the three tabs
- Tables CRUD with zones and the four statuses
- Kitchen display on an ActionCable channel, with the timeout colouring
- POS route with the cart, an IndexedDB catalog cache and an offline queue

Acceptance: I can take a walk-in order on the POS, pay it by cash, see it on
the kitchen display, reopen it, add an item, and the order story shows the
diff and the amount change.

Write the plan first.
```

Two prompts worth keeping for later

- *Audit:* "Review every endpoint that writes stock or money. For each, tell me what happens if it is called twice with the same payload, and what happens if the request dies halfway. Fix anything that is not safe."
- *Parity check:* "Compare the implemented screens against `docs/SPEC.md`. List what is missing, what is partial, and what I built that the spec does not mention."

Gaps and open questions

The review was read-only on a live system, so nothing was created, edited or deleted. That leaves real gaps, listed honestly here rather than guessed at.

Not reachable from the admin panel

- The **desktop POS** is installed Windows software, not a web app, so the till, payment screen, shift close, My Till and the offline queue were seen only through their settings and permissions
- The **tablet app** for order takers, the **rider app** and the **customer website** were likewise not opened

- The **Marketing module** (campaigns, campaign log, contacts, auto-send log) appears in the permission set but has no admin screen; it lives in the POS

Features with nothing configured, so behaviour is known only from the help text

- Loyalty rules, redemption options and points history — zero rules exist
- Promotions — none created
- Vouchers — no batches
- Regular discount tiers — none created
- Purchase orders — the list was empty for the period, though 730 purchases exist historically
- Multi-branch — no linked stores, so cross-outlet behaviour is unconfirmed

Flows that need a write to confirm

- Creating and posting a stock document, and the stock count flow end to end
- Day-End Settlement, which the Cashier Till report depends on
- Running payroll and the advance approval cycle
- The PO receive, return and pay cycle and what it writes to the ledger

Detail not captured

- Exact columns and filters for most of the 27 reports; only Grand Report, Cashier Till, Purchase Price History, Stock in Hand and Deduction Health were opened
- One of the eight Orders permissions and the code for the fourth Suppliers permission
- The finance screens below the dashboard: Combined Ledger, Journal Voucher, Fixed Assets, Liabilities, Equity, Trial Balance, Balance Sheet
- Several settings screens: Order Taker, Customer Fields, POS Notifications, Barcode Scanning, Stock Alerts, Reset Stock, the PIN screens, Signed-In Devices, WhatsApp and Plan

Questions to settle before Phase 1

1. One outlet or several from the start? Multi-outlet is cheap to design in now and expensive to retrofit
2. Is PRA e-IMS filing needed, or is tax display-only for now?
3. Will the POS be a browser PWA, or does it need to be installed software for printer access?
4. Is the fingerprint kiosk a device you already have, and what does it expose — a local API, a file drop, a vendor SDK?

5. Does the first version need the double-entry accounting, or do the account and supplier ledgers cover it?

If you want any of these closed, the quickest route is a second pass with permission to create and then delete test records, or a sandbox account.